# Contents


**1 System Implementation** **2**

1.1 Introduction . . . 2
1.1.1 Problem framing . . . 2
1.1.2 Scope of this chapter . . . 2
1.1.3 Methodology — Zarour (2025) as structuring lens . . . 2
1.2 System at a Glance . . . 3
1.2.1 Host-level architecture . . . 3
1.2.2 Data plane . . . 3
1.2.3 Operational inventory . . . 4
1.2.4 Technology stack . . . 4
1.3 Zarour's Nine MLOps Practices — Implementation . . . 6
1.3.1 P1 — Version control . . . 6
1.3.2 P2 — Continuous integration . . . 6
1.3.3 P3 — Reproducibility . . . 7
1.3.4 P4 — Containerization and infrastructure-as-code . . . 7
1.3.5 P5 — Collaboration and documentation . . . 8
1.3.6 P6 — Continuous training . . . 8
1.3.7 P7 — Experiment tracking and model registry . . . 9
1.3.8 P8 — Continuous monitoring . . . 9
1.3.9 P9 — Feedback loop and drift-triggered retraining . . . 10
1.4 Cross-cutting: CI/CD as the binding fabric . . . 11
1.4.1 PR gate vs deploy gate . . . 11
1.4.2 Tailscale-mediated deploy without exposing the host . . . 12
1.4.3 GIT_SHA propagation as the reproducibility–monitoring link . . . 12
1.4.4 Why a single-host deploy is the right MVP scope . . . 12
1.5 Maturity Assessment and Limitations . . . 13
1.5.1 Practice-by-practice status . . . 13
1.5.2 Quantitative evaluation . . . 14
1.5.3 Known gaps . . . 15
1.5.4 Deferred tooling and adoption triggers . . . 15
1.6 Conclusion . . . 16
.1 Glossary . . . 16

# Chapter 1

# System Implementation

## 1.1 Introduction

### 1.1.1 Problem framing

Electricity price forecasting (EPF) is the task of predicting day-ahead wholesale electricity prices for a given bidding zone from a combination of historical prices and exogenous signals such as load, renewable generation, and calendar effects. Research on EPF has produced increasingly accurate offline benchmarks — LEAR, deep neural networks, and hybrid ensembles — evaluated on fixed historical holdout periods under idealised conditions.

Production EPF, however, imposes a different and harder set of requirements. A deployed forecasting system must continuously ingest fresh market data from external APIs, retrain or recalibrate models on a rolling basis as the price distribution shifts, monitor prediction quality in near-real time, and align the evaluation methodology with actual operational use — a contrast that the EPF research literature largely sidesteps in favour of reporting accuracy on offline benchmarks [5].

### 1.1.2 Scope of this chapter

This chapter documents the *system* built around the forecasting models: the orchestration layer, infrastructure choices, observability stack, and CI/CD pipeline that together enable the platform to run reliably in production. It does not cover the modelling research itself — LEAR hyperparameters, DNN architecture, the asinh price transformation, or the Diebold–Mariano and Giacomini–White statistical tests — except where a modelling decision directly motivates or constrains an MLOps practice. Operational how-to material such as running backfills, adding new bidding areas, or re-deploying Prefect flows is maintained in the repository's `RUNBOOK.md` and `CONTRIBUTING.md` and is not repeated here.

### 1.1.3 Methodology — Zarour (2025) as structuring lens

The chapter is organised around the nine MLOps practices identified in the systematic literature review by Zarour et al. [9]: version control, continuous integration, reproducibility, containerisation and infrastructure-as-code, collaboration and documentation, continuous training, experiment tracking and model registry, continuous monitoring, and the feedback loop with drift-triggered retraining. Each practice section follows a fixed shape: (i) Zarour's own framing of the practice and what it requires of a production ML system; (ii) the concrete implementation found in algo-repo; (iii) the technology chosen and the alternatives that were considered and rejected; (iv) the design rationale given the single-host MVP scope of the platform; and (v) an optional architecture or flow diagram where one aids comprehension. This structure makes it straightforward to assess

which practices are fully implemented, partially implemented, or deliberately deferred, and to trace every technology choice back to an explicit justification.

## 1.2 System at a Glance

Before examining each Zarour practice in detail, this section presents the platform as a whole: its host-level topology, the end-to-end data plane, the complete inventory of scheduled deployments, and the technology choices that underpin every layer.

### 1.2.1 Host-level architecture

The platform runs entirely within a single Docker Compose stack on one dedicated host, keeping operational complexity proportionate to the MVP scope. Five long-lived services form the persistent backbone of that stack: ClickHouse, which stores all time-series data and ML artifacts; the Prefect server, which exposes the orchestration API and web UI; the Prefect worker, which polls for scheduled runs and manages a Docker work pool; Loki, which receives structured log streams from Python via `python-logging-loki`; and Grafana, which visualises both ClickHouse metrics and the Loki log stream through file-provisioned dashboards. When the Prefect worker dispatches a scheduled flow run it spawns an ephemeral container from the `algo-repo:latest` image; that container reads from and writes to ClickHouse, calls the ENTSO-E Transparency Platform API, and emits structured logs back to Loki before being automatically removed on completion.

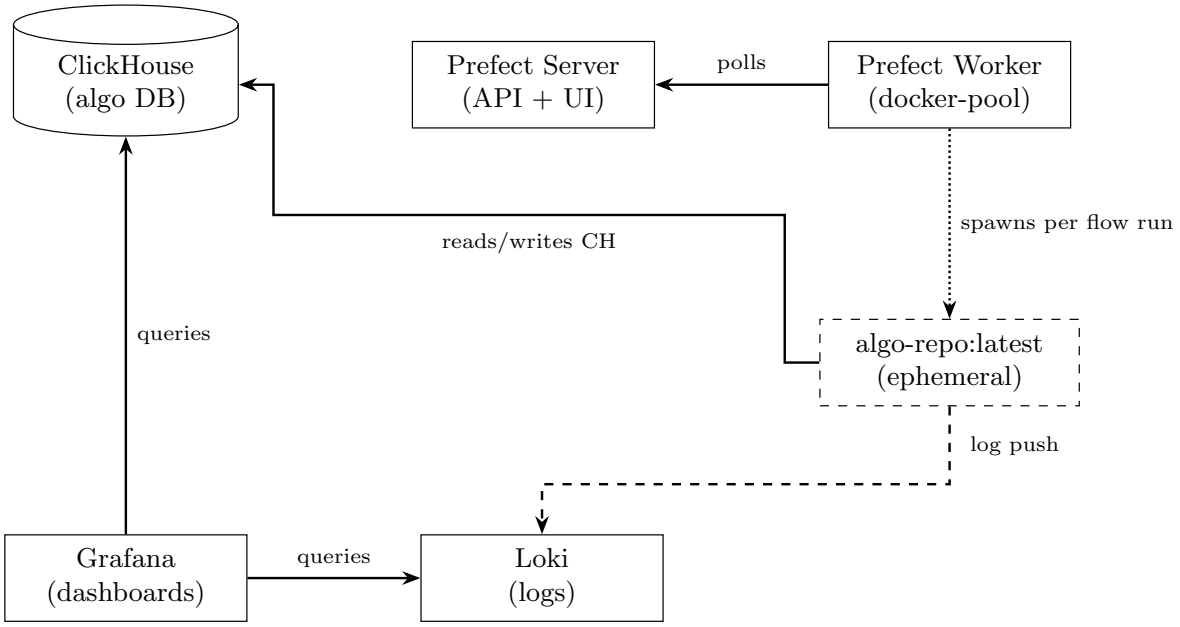



Figure 1.1: Host-level architecture. A single Docker Compose host runs ClickHouse, the Prefect server and worker, Loki, and Grafana. Flow runs spawn ephemeral `algo-repo:latest` containers that read ClickHouse, call ENTSO-E, and emit logs back to Loki.

### 1.2.2 Data plane

Data enters the platform through the ENTSO-E Transparency Platform API, where four fetch functions retrieve energy prices, total-load forecasts, generation forecasts, and wind and solar forecasts for each configured bidding area. The raw XML responses are parsed by typed

mapper functions that produce `Observations` and `Forecast` domain objects, which are then ingested into ClickHouse tables `algo.timeseries` and `algo.forecasts` via Arrow-batch inserts. Both tables use `ReplacingMergeTree` engines, which deduplicate lazily on merge; read paths therefore either append the `FINAL` keyword to their queries or call `drop_duplicates` on the loaded DataFrame to ensure correctness. A `DataPreprocessor` loads this data, pivots it to a daily-wide feature matrix through a configurable sequence of `FeatureTransformer` objects, and feeds it to either `BacktestEngine` for offline evaluation or `LiveRunner` for day-ahead production forecasting. Predictions and trained model artifacts are persisted to `algo.experiment_runs`, `algo.experiment_predictions`, and `algo.model_artifacts`, from where the reconciliation flow back-fills actuals the following day and the drift-detection flow compares the recent price distribution against a ninety-day reference window, triggering a Slack alert and an optional automatic retraining run when significant drift is detected.

### 1.2.3 Operational inventory

Table 1.1 lists every Prefect deployment currently registered in the system, together with its schedule, the flow it executes, and the Zarour practice it realises.

Table 1.1: Operational inventory of Prefect deployments. Cron strings are UTC.

| **Deployment** | **Area** | **Cron (UTC)** | **Flow module** | **Purpose** | **Zarour** |
|---|---|---|---|---|---|
| entsoe-backfill-dk1 | DK1 | 0 * * * * | schedules.ingestion. entsoe_backfill_flow | Hourly ENTSO-E ingest | P1, P2 |
| entsoe-backfill-dk2 | DK2 | 0 * * * * | *(as above)* | Hourly ENTSO-E ingest | P1, P2 |
| entsoe-backfill-nl | NL | 0 * * * * | *(as above)* | Hourly ENTSO-E ingest | P1, P2 |
| entsoe-backfill-be | BE | 0 * * * * | *(as above)* | Hourly ENTSO-E ingest | P1, P2 |
| entsoe-backfill-de | DE | 0 * * * * | *(as above)* | Hourly ENTSO-E ingest | P1, P2 |
| entsoe-backfill-fr | FR | 0 * * * * | *(as above)* | Hourly ENTSO-E ingest | P1, P2 |
| entsoe-backfill-pl | PL | 0 * * * * | *(as above)* | Hourly ENTSO-E ingest | P1, P2 |
| daily-forecast-dk1_lear | DK1 / LEAR | 0 6 * * * | schedules.forecast. daily_forecast_flow | Daily refit and day-ahead forecast | P6 |
| daily-forecast-de_lear | DE / LEAR | 0 6 * * * | *(as above)* | Daily refit and day-ahead forecast | P6 |
| reconcile-predictions | all | 0 18 * * * | schedules.reconciliation. reconcile_predictions_flow | Day-after reconciliation against published actuals | P7, P8 |
| drift-detection-dk1 | DK1 | 0 19 * * * | schedules.drift. drift_detection_flow | 90-day reference vs 7-day current, KS test | P8, P9 |
| drift-detection-de | DE | 0 19 * * * | *(as above)* | 90-day reference vs 7-day current, KS test | P8, P9 |

### 1.2.4 Technology stack

The table below records each tool's role, whether it was already present at the start of the MLOps build-out or was added during it, and the one-line rationale for the choice.

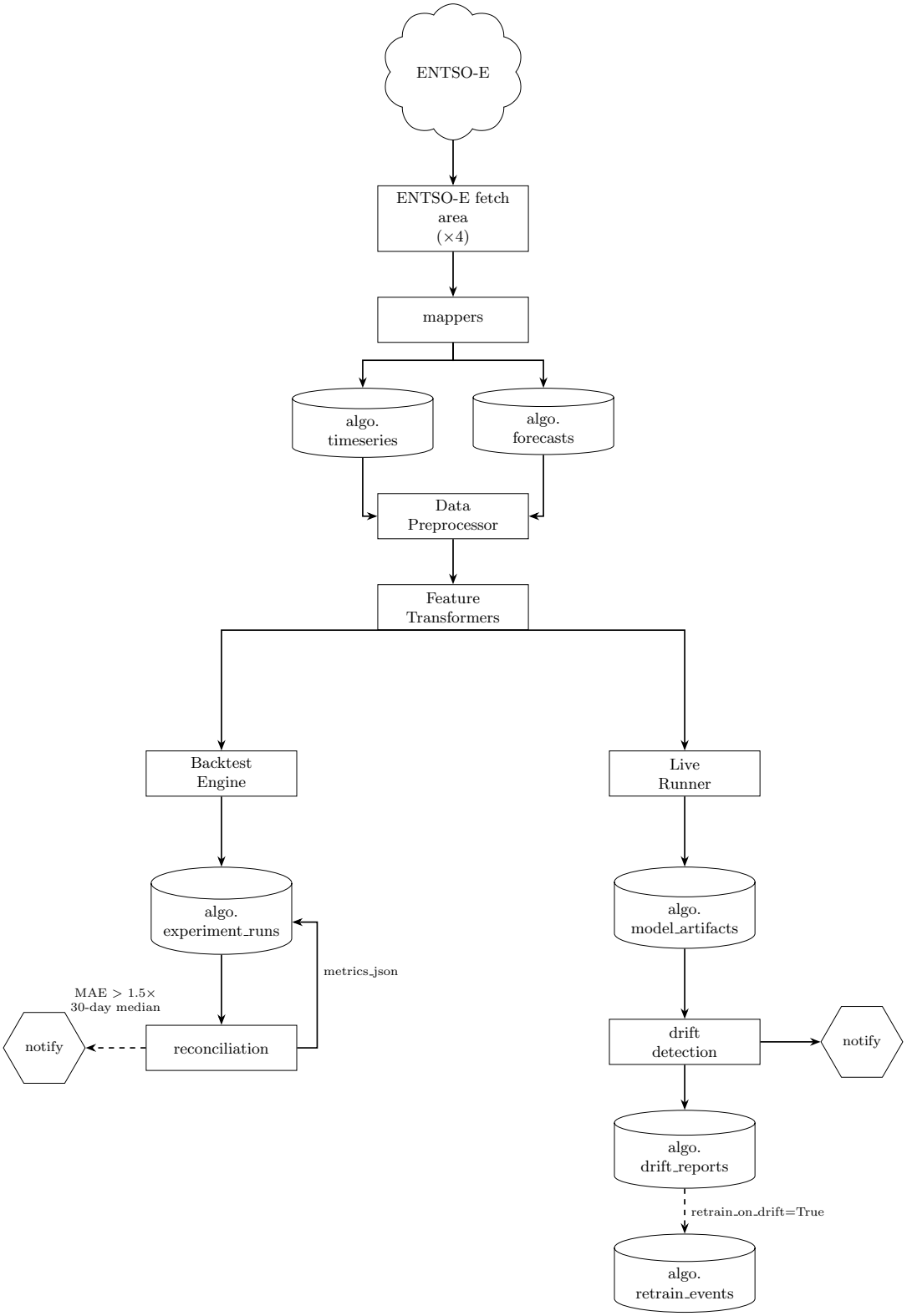



Figure 1.2: Data plane. ENTSO-E data flows through typed mappers into ClickHouse, where the forecasting pipeline reads it, trains and evaluates models, and writes predictions and artifacts back. Reconciliation and drift detection close the loop on a daily cadence.

## 1.3 Zarour's Nine MLOps Practices — Implementation

The nine sections that follow map every MLOps practice identified by Zarour et al. (introduced in §1.3) to a concrete subsystem in algo-repo. Each section follows the same four-paragraph shape: the practice's framing, the implementation found in this repository, the technology chosen and its rejected alternatives, and the design rationale scoped to a single-host MVP. Where a practice is only partially realised, the limitation is stated explicitly and cross-referenced to §5.

### 1.3.1 P1 — Version control

Version control is the foundational practice under which all code, data schemas, and configuration that affect model behaviour are tracked in a source-control system, enabling reproducible reconstruction of any past system state [9]. Zarour et al. treat it as a prerequisite for every other practice, because without a shared history of code and configuration changes, claims about model reproducibility or deployment rollback cannot be substantiated.

The platform uses Git with GitHub as the remote. Every artefact that can affect a model run is versioned: Python experiment configs in `configs/` are first-class Git files whose history records every calibration-window or scaler change; ClickHouse schema changes live in numbered SQL migration files under `src/infrastructure_clickhouse/migrations/` that are auto-discovered in sorted order by `run_migrations.py` and never modified retroactively; Grafana dashboard definitions are maintained as JSON under `grafana/dashboards/` and alert rules are file-provisioned from `grafana/provisioning/alerting/rules.yaml`, so neither is ever edited through the UI and both are subject to code review before taking effect.

Git was chosen over Mercurial or Perforce on the grounds that it is the industry-standard choice for the GitHub ecosystem the project already inhabits. DVC was evaluated for data versioning and rejected: ENTSO-E data is replayable in full from the public API, and combining ClickHouse's `ReplacingMergeTree` engine with the `GIT_SHA` column on `algo.experiment_runs` provides sufficient provenance without maintaining a separate binary store.

Versioning every artefact that affects model output — configs, migrations, dashboards, and alert rules — means a single `git log` spans the experiment, the infrastructure, and the alerting policy together, which closes the gap between operational changes and the audit trail expected of a production ML system.

### 1.3.2 P2 — Continuous integration

Continuous integration, as characterised by Zarour et al., requires that every proposed change be automatically validated against a test suite and a set of code-quality checks before it is permitted to reach the main branch [9]. The practice prevents the accumulation of silent regressions and ensures that the repository always remains in a deployable state.

Three GitHub Actions workflows gate every pull request. `.github/workflows/pr_test.yml` runs the full pytest suite, which mirrors the `src/` layout and covers domain models, XML mappers, ClickHouse ingest, and the forecasting pipeline including the LEAR smoke test in `tests/price_forecasting/test_smoke.py`. `.github/workflows/pr_ruff-lint.yaml` runs `ruff check` for linting and import hygiene. `.github/workflows/pr_smoke.yml` runs an end-to-end LEAR smoke test against a minimal dataset; this workflow includes a path filter introduced on the current branch that skips the smoke step when a push touches only files outside the gated paths, preventing documentation-only or other unrelated changes from being held back by a several-minute Docker build.

GitHub Actions was chosen over Jenkins or CircleCI because it is co-located with the repository and requires no separately managed runner host. The three workflows are kept as separate files rather than a single monolithic workflow so that the smoke test — the slowest step — can be skipped or re-run in isolation without re-triggering linting.

Separating the smoke test from the unit and lint checks resolves the tension between fast feedback on small changes and thorough validation of code that touches the forecasting pipeline, keeping both workflows proportionate to the kind of change being reviewed.

### 1.3.3 P3 — Reproducibility

Reproducibility in the context of MLOps requires that a past model result can be reconstructed from the same code version, data window, and hyperparameters, without relying on implicit runtime state or untracked configuration [9]. Zarour et al. identify it as one of the most frequently cited challenges in practitioner surveys, because the combination of non-deterministic training, mutable data sources, and absent provenance records makes debugging production regressions extremely difficult.

The platform addresses reproducibility along two axes. First, every row in `algo.experiment_runs` carries `git_sha`, `data_start_utc`, and `data_end_utc`, providing a minimal sufficient provenance triple that identifies both the code version and the data window used for any given run. Second, paper-correct calibration windows are fixed in `configs/dk1_lear.py` and `configs/de_lear.py`, and the `AsinhScaler` variance-stabilising target transform is selected via `ExperimentConfig.target_scaler`, in accordance with the recommendation by Lago et al. [5] for EPF systems where the price series contains negative values and extreme spikes. The DNN ensemble averages `n_dnn_ensemble` replicates with distinct, enumerated random seeds, so the trained ensemble is fully described by the config and a deterministic seed sequence with no untracked randomness.

The `GIT_SHA` environment variable is baked into the production image at build time using `-build-arg GIT_SHA=$(git rev-parse HEAD)` in `cd_deploy.yml`, rather than being resolved at runtime via a `git rev-parse` call. This is necessary because the deploy host holds only the built image, not the source tree. Local runs emit an empty string so that a dirty working tree never stamps experiment rows with a misleading SHA.

Baking the SHA into the image rather than resolving it at container startup resolves the conflict between a deployment model that has no source checkout on the host and the need for every experiment row to carry an unambiguous code identifier. Together with the data window columns, it makes every `algo.experiment_runs` row self-describing.

### 1.3.4 P4 — Containerization and infrastructure-as-code

Zarour et al. define containerization and infrastructure-as-code as the practice of packaging every component of the ML system — including all of its runtime dependencies — into reproducible container images, and of declaring the full operational topology in version-controlled configuration files rather than in hand-applied host commands [9]. The goal is that the infrastructure can be rebuilt from scratch from the repository alone.

The platform's entire operational topology is declared in `docker-compose.yml`, which defines the five long-lived services (ClickHouse, Prefect server, Prefect worker, Loki, Grafana) together with their volume mounts, environment variable sources, and network configuration. Flow-run containers are built from `Dockerfile` (the application image) and the Prefect worker from `Dockerfile.worker`. Grafana's data-source and dashboard-provider configuration live in `grafana/provisioning/`, and alert rules are declared in `grafana/provisioning/alerting/rules.yaml`. All of these files are tracked in Git and applied by restarting the stack — no manual UI operations are required or permitted. The Prometheus scrape configuration lives at `prometheus/prometheus.yml` and is mounted read-only into the Prometheus container; the same file-provisioning discipline that applies to Grafana dashboards and alert rules applies here — the scrape policy is reviewed in Git and never edited at runtime.

Docker Compose was chosen over Kubernetes for this deployment. The single-host scope makes the added complexity of Kubernetes — a control plane, etcd, node pools, and Helm charts

— unjustified at MVP scale. Kubernetes is listed as a deferred tool with a clear adoption trigger: the moment a second deploy host becomes necessary. Helm and k3s were also considered and rejected on the same grounds.

Declaring the alerting policy in a file-provisioned YAML rather than through the Grafana UI closes the loop with P1: the rules that govern when the on-call team is notified are reviewable artefacts in Git, subject to the same pull-request gate as model configs and schema migrations. A container restart reapplies the policy from the file, so UI drift is structurally impossible.

### 1.3.5 P5 — Collaboration and documentation

Collaboration and documentation, as Zarour et al. frame it, encompasses the practices that allow a team to work on a shared ML system without relying on tribal knowledge: conventions files, onboarding guides, runbooks for operational incidents, and tooling that surfaces system events to team members proactively rather than requiring them to poll dashboards [9]. The practice is particularly important in ML systems because the surface of implicit knowledge is larger than in conventional software — calibration window choices, scaler semantics, and `ReplacingMergeTree` deduplication behaviour all require explicit documentation to be reproducible across team members.

The repository carries four documentation artefacts. `CLAUDE.md` records project conventions — naming rules, CLI flag order, circular-import constraints, and design rationale — and is written to be machine-readable by LLM coding agents as well as by human contributors. `docs/REPO_OVERVIEW.md` provides a deep reference of the full system for any reader coming in cold. `docs/RUNBOOK.md` covers manual interventions and incident triage. `docs/CONTRIBUTING.md` documents the branching policy, PR conventions, and the procedure for onboarding a new bidding area. On the alerting side, drift events and accuracy-degradation alerts post to a Slack webhook via `schedules.alerting.notify`, ensuring that the team is notified of production anomalies even when nobody is watching the Grafana dashboards.

Slack webhooks were chosen over PagerDuty or Opsgenie because the team is small enough that a single notification channel is sufficient — there is no on-call rotation that would benefit from escalation policies or page acknowledgement workflows. The `notify` helper is a safe no-op when `SLACK_WEBHOOK_URL` is unset, so local development and CI runs never fail due to a missing webhook.

Splitting the documentation into two distinct files — `CLAUDE.md` for conventions and `REPO_OVERVIEW.md` for reference — serves two different readers: LLM agents need a terse, rule-dense conventions file they can load as context without scanning prose, while human contributors benefit from the longer narrative reference. Keeping both files short and load-bearing reduces the risk that documentation drifts out of date with the codebase.

### 1.3.6 P6 — Continuous training

Continuous training requires that the model be retrained automatically on a regular cadence so that it always reflects the most recent data distribution, rather than drifting against a fixed snapshot trained at deployment time [9]. Zarour et al. distinguish it from one-off retraining events and from the reactive drift-triggered path covered by P9: the core requirement is a scheduled, unconditional refit that runs regardless of whether drift has been detected.

The `daily-forecast-dk1_lear` and `daily-forecast-de_lear` Prefect deployments execute the `schedules.forecast.daily_forecast_flow` flow at 06:00 UTC every day. Each run calls `LiveRunner.run_live_with_metadata()`, which loads the latest ClickHouse data window, refits the full ensemble of calibration-window models from scratch, and immediately produces a day-ahead forecast. For DNN configurations, `LiveRunner` fits `n_dnn_ensemble` replicates with distinct random seeds and averages their outputs before recording the result, following the ensemble-averaging approach recommended by Lago et al. [5]. The trained models are serialised

to `algo.model_artifacts` via `forecast_cli.tracking` so that the artefact is always co-located with its experiment row.

A full refit from scratch on each daily run was chosen over incremental or warm-start learning. The daily data increment is small relative to the calibration window lengths (56–1456 days), so the marginal cost of a clean refit is low and the benefit — no accumulated approximation error from incremental updates — is material. Keeping training inside the same Prefect flow as prediction means the trained ensemble is always consistent with the forecast it produced.

Combining the training and prediction steps in a single `daily-forecast` flow means that a task-level retry covers both phases with a single retry policy. A separate predict deployment would be cheaper per run but would require coordination to ensure the serving model matches the most recent training state, which introduces a failure mode absent in the co-located design.

### 1.3.7 P7 — Experiment tracking and model registry

Zarour et al. define experiment tracking and model registry as the practice of recording the full provenance of every model run — hyperparameters, data windows, performance metrics, and the model artefact itself — in a queryable store, so that past results can be reproduced, compared, and promoted to production without relying on ad-hoc file management [9]. They note that the registry role and the tracking role are often conflated but serve different consumers: tracking serves the data scientist reviewing experiment history, while the registry serves the deployment pipeline that needs a canonical "latest approved model" pointer.

Both roles are handled in ClickHouse. `algo.experiment_runs` stores one row per run with columns `training_duration_seconds`, `data_start_utc`, `data_end_utc`, `model_artifact_id`, and `git_sha`, making every run self-describing. `algo.experiment_predictions` stores one row per period per day for downstream reconciliation. `algo.model_artifacts` stores base64-encoded joblib blobs, keeping the artefact attached to its experiment row via `model_artifact_id`. `algo.hyperparam_trials` persists every Optuna trial from `forecast-cli tune` runs, enabling resumable search across sessions. On the research surface, W&B serves two projects: `algo-research` for backtests, ablation sweeps, and Optuna sweeps, and `algo-prod` for production reconciliation sync — one continuous W&B run per `(area, config_name)` pair, with one step per reconciled day. `WANDB_MODE=disabled` no-ops the tracker cleanly in CI and local development.

MLflow was evaluated as an alternative and deferred until the experiment-config count exceeds ten. At the current scale, ClickHouse and W&B together cover both the audit-log role and the research-surface role that MLflow would fill, without adding a seventh service to the Compose stack. A filesystem model store was rejected: storing artefacts as blobs in `algo.model_artifacts` keeps them attached to the experiment row rather than in a path that must be separately managed and backed up.

Using one OLAP store for predictions, metrics, and artefacts means that day-after reconciliation is a single SQL query against `algo.experiment_predictions` rather than a join across systems. The `ReplacingMergeTree(created_utc)` engine on `algo.experiment_runs` makes the reconciliation update a safe re-insert rather than a mutable `UPDATE`, which is consistent with the append-only semantics used throughout the platform.

### 1.3.8 P8 — Continuous monitoring

Continuous monitoring, as described by Zarour et al., requires instrumenting the production ML system across at least three axes — data quality, model performance, and system health — so that degradation is detected automatically and surfaced to the team without manual inspection [9]. The practice closes the gap between a model that performs well at training time and one that continues to perform well as the environment changes around it.

Structured log events from every Prefect flow and infrastructure module are pushed directly to Loki via the `python-logging-loki` handler configured in

infrastructure_clickhouse.logging_setup. No Promtail sidecar is used. Three Grafana dashboards auto-provision from grafana/dashboards/: the Data Quality dashboard covers coverage and freshness of algo.timeseries and algo.forecasts; the Operational dashboard presents the Loki log stream filtered by job=algo-repo; and the System Health dashboard (introduced in §3.8.x above) covers ClickHouse and Grafana liveness via Prometheus. Model-quality metrics — MAE, RMSE, and sMAPE per reconciled day, per area, per config — do not live in Grafana; they are owned by the W&B algo-prod project, which the dual-write tracker in observability_wandb updates from schedules.reconciliation. The split reflects a deliberate boundary: Grafana hosts platform-and-money concerns, W&B hosts model-quality concerns, and the two surfaces read from the same ClickHouse tables for the facts they care about. File-based alert rules in grafana/provisioning/alerting/rules.yaml define three conditions: data-staleness fires when algo.timeseries has not received a new row in twelve hours; drift-detected-today fires when any row in algo.drift_reports for the current day carries drift_detected = 1; and reconciliation-missed fires when no reconcile-predictions run has completed in the last 25 hours. In addition, schedules.reconciliation._maybe_alert_accuracy_degradation fires a Slack notification whenever today's MAE exceeds $1.5\times$ the 30-day median, computed from algo.experiment_runs via queries.get_recent_accuracy. A fourth dashboard, *System Health*, is backed by Prometheus, which scrapes ClickHouse's native /metrics endpoint, Grafana's /metrics endpoint, and itself on a 15-second cadence and retains 120 days of time-series data. The up{} metric exposed by every scrape target is the platform's primary liveness signal: it drives the new clickhouse-down alert rule.

Loki was chosen over the ELK stack (Elasticsearch, Logstash, Kibana) because it is a single binary with substantially lower memory and operational overhead, which is proportionate to the single-host deployment. Promtail was rejected in favour of direct Python push, which removes a moving part from the logging pipeline and means no log events are silently dropped due to a misconfigured sidecar. Prometheus sits alongside Loki rather than replacing it: Loki is the logs tier, Prometheus the metrics tier. They are complementary, and Grafana remains the single dashboard and alerting surface above both.

Three axes of monitoring — data, model, and system — match the decomposition in Zarour's P8 framing and ensure that an alert is generated at whichever layer first shows a problem. Keeping alert rules in a file-provisioned YAML closes the loop with P4: the alerting policy is reviewed in Git and reapplied by a container restart, so the live alerting configuration always matches the committed file.

### 1.3.9 P9 — Feedback loop and drift-triggered retraining

The feedback loop practice, as framed by Zarour et al., requires that the production system detect when model inputs or outputs have shifted sufficiently to degrade prediction quality, and respond automatically by triggering a retraining run rather than waiting for a human to notice the degradation on a dashboard [9]. They identify the absence of an automated feedback loop as a common maturity gap — systems that monitor drift but require a manual retrain decision provide weaker guarantees than systems that close the loop entirely.

Drift detection runs in-process inside the drift-detection-dk1 and drift-detection-de Prefect deployments via Evidently's DataDriftPreset, which applies a Kolmogorov–Smirnov test comparing a 90-day reference window against a 7-day current window on the price series for each area. Drift results — score, threshold, test name, and the boolean drift_detected flag — are written to algo.drift_reports. A detected drift event posts a warning to Loki and a Slack notification via schedules.alerting.notify. When a config carries retrain_on_drift=True, the flow additionally enqueues a triggered_retrain_flow which writes a running audit row to algo.retrain_events, calls daily_forecast_flow synchronously, and closes the row with a completed or failed status and a strictly later updated_utc so the ReplacingMergeTree(updated_utc) engine retains the terminal row.

Evidently was deployed in in-process mode rather than as a separate Evidently service. Running the drift check inside the Prefect flow envelope means the check inherits the flow's task-level retry policy and is trivially cancelled or re-run via the Prefect UI without managing an additional service endpoint. Arize and WhyLabs were evaluated and rejected as cloud-hosted monitoring platforms whose external-service dependency is unjustified at single-host MVP scope.

Making the retrain trigger conditional on a `retrain_on_drift` config flag rather than always-on ensures that every retraining event is auditable: each triggered run has a corresponding row in `algo.retrain_events` that links the run to the specific `drift_report_id` that caused it, so the full causal chain from detected drift to updated model is recoverable from ClickHouse alone.

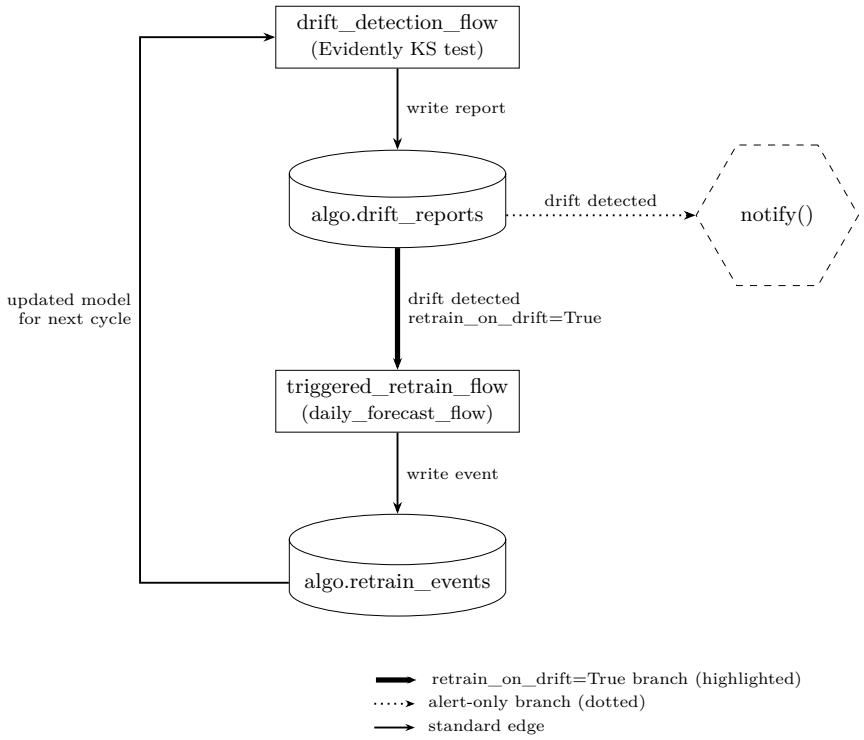



Figure 1.3: Drift-triggered retraining loop. Drift events land in `algo.drift_reports` and post to Slack; configs with `retrain_on_drift=True` additionally trigger a retraining flow that records each event in `algo.retrain_events`.

## 1.4 Cross-cutting: CI/CD as the binding fabric

CI/CD in algo-repo is not a single pipeline but a layered system of gates: three workflows that protect every pull request from landing broken code, and one deploy workflow that ensures every merge to `main` propagates faithfully to the running production host. Together they make the repository and the host converge reliably without requiring manual intervention, which is the property that all of the observability and tracking practices built on top of this foundation depend on.

### 1.4.1 PR gate vs deploy gate

Three GitHub Actions workflows form the PR gate. `pr_test.yml` runs the full pytest suite — domain models, XML mappers, ClickHouse ingest, and the LEAR smoke test in `tests/price_forecasting/test_smoke.py` — against every pull request targeting `main`.

pr_ruff-lint.yaml enforces ruff check for linting and import hygiene. pr_smoke.yml builds the algo-repo:latest image, starts a Compose stack with ClickHouse and a Prefect server and worker, and executes an end-to-end daily-forecast and reconciliation run against synthetic data; a path-filter step introduced on the current branch skips the expensive smoke steps entirely when a push touches only files outside the gated paths, so docs-only commits pass the required check without waiting for a multi-minute Docker build. The deploy gate, cd_deploy.yml, fires on every merge to main and serves a different purpose: it joins the production host over the Tailscale network, pulls the merged code, runs ClickHouse migrations, rebuilds algo-repo:latest with GIT_SHA baked in via -build-arg GIT_SHA=$(git rev-parse HEAD), reruns both deploy_ingestion.py and deploy_forecast.py to register updated Prefect deployments, and finally triggers daily-forecast-dk1_lear and polls until the flow run completes, confirming that the freshly deployed image can produce a valid forecast on the live host. A parallel reload_grafana job restarts the Grafana container so that file-provisioned dashboards and alert rule changes take effect, and a separate main_cache_warm.yml workflow runs on merges to main to pre-populate the GitHub Actions cache used by the next PR's Docker build. The PR gate enforces correctness; the deploy gate enforces delivery.

### 1.4.2 Tailscale-mediated deploy without exposing the host

The deploy host accepts inbound connections only over the Tailscale mesh network, which means the public internet sees no open SSH port on it at all. The alternative approaches — a publicly accessible SSH endpoint, a bastion host, or a VPN appliance — each require either that the host exposes a port reachable by an attacker who can guess or enumerate addresses, or that a separate jump host be maintained and kept patched. Tailscale replaces both with a WireGuard mesh where every device authenticates via the Tailscale coordination server before a packet is accepted; the deploy host's firewall drops everything not originating on the tailnet, and the GitHub Actions step that needs to SSH in runs tailscale/github-action@v4 to join the same network for the duration of the workflow run. The TS_AUTHKEY secret is an ephemeral pre-authenticated key scoped to the CI device tag, so a leaked key cannot create a persistent backdoor. cd_deploy.yml joins the tailnet once per job, executes the SSH commands, and the connection is torn down when the runner exits; no permanent network path to the host is ever open to the public internet.

### 1.4.3 GIT_SHA propagation as the reproducibility–monitoring link

Every row that cd_deploy.yml produces in algo.experiment_runs carries the same git_sha that was baked into the image during the deploy-gate build step: the SHA of the exact commit that was merged to main. The Grafana Operational dashboard queries the Loki log stream filtered by job=algo-repo and presents structured log events that include the same SHA in their fields, because every flow container is built from that same image. This means that if the W&B algo-prod run for a given (area, config_name) shows an unexpected MAE spike beginning on a particular day, the Grafana Operational dashboard will show the Loki events from the same period alongside the git_sha of the image that produced them — making it possible to git log back to the exact commit that introduced the regression. This closes the loop between P3 (reproducibility) and P8 (continuous monitoring): the audit trail for a production anomaly runs from the Grafana panel, through the algo.experiment_runs row, to a specific, checkable commit in the Git history without any gap in the chain.

### 1.4.4 Why a single-host deploy is the right MVP scope

Running the entire stack — ClickHouse, Prefect server, Prefect worker, Loki, Grafana, and the ephemeral flow containers — on a single Docker Compose host means there is exactly one failure domain to reason about: if the host goes down, all services go down together and

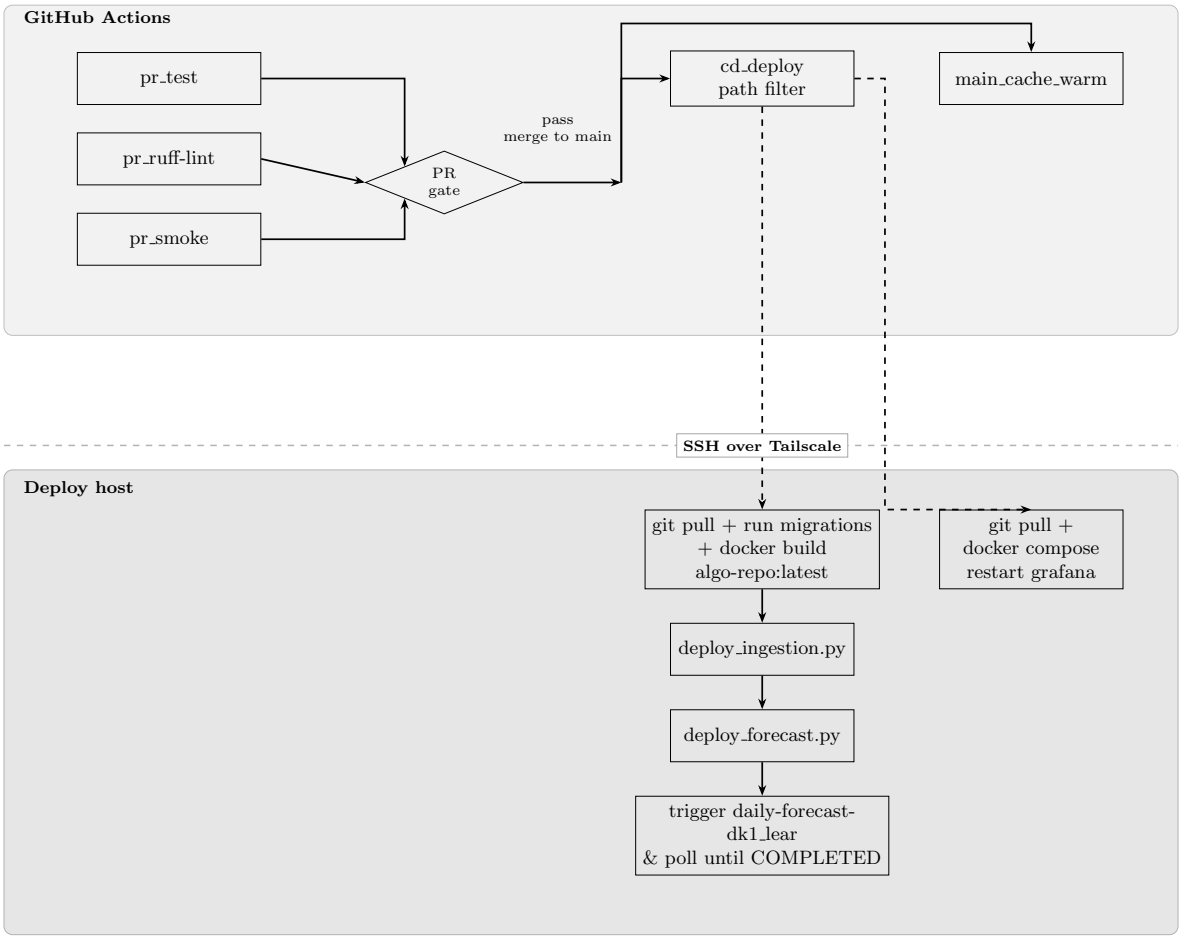



Figure 1.4: CI/CD pipeline. GitHub Actions runs the PR gate and the deploy gate; the deploy gate reaches the production host across the Tailscale link, rebuilds the image, redeploys Prefect, and triggers a smoke forecast.

are restored together by restarting the Compose stack, which is the correct blast radius for a single-operator MVP. Kubernetes, as observed by Zarour et al. with respect to containerisation maturity levels [9], becomes the appropriate choice when workloads must be distributed across multiple hosts and when the overhead of a control plane and an etcd cluster is justified by the availability requirements of the system; neither condition holds here. Multi-host deployment would introduce the need for a service mesh or a shared storage layer for ClickHouse, a distributed Prefect work pool, and cross-host log aggregation — complexity that would slow down iteration without improving the platform's ability to deliver daily forecasts reliably. The single-host scope also simplifies rollback: reverting to a previous image tag is a single `docker build` followed by a `docker compose restart`, with no pod drain, replica set update, or rolling-deployment coordination required. The right adoption trigger for Kubernetes is the appearance of a second deploy host, not a target architecture chosen speculatively.

## 1.5 Maturity Assessment and Limitations

### 1.5.1 Practice-by-practice status

The table below summarises the implementation status of each Zarour practice as realised in algo-repo. The three status labels map directly to the evidence inventory that precedes them: a

practice is *Realised* when a concrete, production-running artefact closes it end-to-end; *Minimal* when the mechanism exists but its scope is intentionally restricted; and *Deferred* when a gap was identified, a tool evaluated, and adoption postponed to a named trigger.

### 1.5.2 Quantitative evaluation

The qualitative maturity table above records whether a practice is realised, not by how much. This section adds quantitative evidence. Following the single-case study guidelines of Runeson and Höst [7], the system is evaluated from its source code, version-control and CI history, and short targeted experiments; the reported figures are indicative of the system over the study window rather than claims about long-run production operation. This framing matches comparable MLOps evaluations in the electricity-forecasting domain [8].

#### ML Test Score

We self-administer the ML Test Score rubric of Breck et al. [1], which scores 28 tests across four sections (data, model development, ML infrastructure, monitoring) on a $\{0, 0.5, 1\}$ scale; the final score is the *minimum* of the four section sums, so the weakest area bounds the result. Table 1.4 reports the per-section sums and the final score.

The binding section is data testing: the system enforces strict input schemas and tests its feature-transformer code, but does not formally document the feature-benefit, feature-cost, or privacy analyses that the rubric also rewards. We read this as the system's primary remaining investment area and revisit it under §5.3.

#### Reproducibility

Reproducibility is measured two ways. First, as provenance coverage: of the 15 live runs in the window, 66.7 % carry the complete provenance triple (`git_sha`, the data window (`data_start_utc`, `data_end_utc`), and `model_artifact_id`); the shortfall is attributable to early runs that predate the CD pipeline baking `GIT_SHA` into the image, whereas the data-window and artefact columns are populated for every run. Second, as a worked re-run: using only a logged run's committed configuration and target date, we regenerated the forecast with `scripts/report/reproduce_run.py` and compared it to the prediction stored in `algo.experiment_predictions`. For the deterministic LEAR configuration on 2026-03-22, the maximum absolute difference across all 24 delivery periods was $7 \times 10^{-6}$ EUR/MWh — five orders of magnitude below any economically meaningful price unit, i.e. numerically identical up to solver-level floating-point tolerance. Under the ACM Artifact Review and Badging terminology the artefact is Available, Functional, and Reusable by self-review; we do not claim the independent "Reproduced" badge, which requires an external reviewer [4].

#### Delivery throughput

Treating each successful run of `cd_deploy.yml` on `main` as one deployment, we report the two DORA *throughput* metrics [2]: over the window there were 19 successful deployments (1.48 per week), with a PR-merge-to-deploy-start lead time of 0.1 min at both the median and the 95th percentile. We deliberately do *not* report change failure rate or mean time to restore: both require an incident corpus that a study window of this length does not provide, and we do not assign a DORA performance tier, whose thresholds are re-derived annually. The sub-minute lead time reflects that `cd_deploy.yml` fires immediately on push to `main` and that the interval measured is from the merge commit to the workflow's own start, not from the first commit on a feature branch.

**Drift detection as monitoring, not a retraining trigger**

The system refits every model from scratch daily on a rolling window at a cost of seconds (§P6), so the model is never stale between scheduled refits. In this regime, drift-*triggered* retraining adds no freshness benefit over the scheduled refit: the cost–benefit analysis of Mahadevan and Mathioudakis [6] shows that triggered retraining earns its keep only when retraining is expensive, and that distribution-drift triggers otherwise incur unnecessary retrains from false positives. We therefore retain drift detection as a *monitoring and alerting* signal rather than a retraining trigger, which is why `retrain_on_drift` defaults to false. This conclusion is conditional: it holds while retraining stays cheap and the rolling window short, and would be revisited for expensive multi-year transformer retraining. We also note a calibration limitation: over the window the detector flagged drift on 35 of 35 reports, a detection rate that is uninformatively high and is listed as future work in §5.3.

**Summary**

Taken together, these measurements turn the chapter's qualitative "Realised" labels into numbers: a single ML Test Score with a per-section breakdown, a provenance-coverage percentage backed by a live re-run that reproduced a logged forecast to within $10^{-5}$ EUR/MWh, and delivery-throughput figures. Each is recomputable by re-running `scripts/report/extract_metrics.py` and `scripts/report/reproduce_run.py` against the live system.

### 1.5.3 Known gaps

Forecast and drift-detection deployments are currently scoped to two bidding areas — DK1 and DE. The five remaining ingestion areas (DK2, NL, BE, FR, PL) receive hourly ENTSO-E data through the `entsoe-backfill-*` deployments but have no corresponding `daily-forecast` or `drift-detection` flow, which means prediction-quality monitoring and drift-triggered retraining are unavailable for those markets. Expanding coverage requires adding new experiment config files and appending the corresponding entries to the `FORECAST_CONFIGS` list in `deploy_forecast.py`; no flow code needs to change.

Model artifacts are serialised with joblib and stored as base64-encoded blobs in the `algo.model_artifacts` ClickHouse table. This is practical for the current LEAR models, whose serialised size is on the order of one hundred kilobytes, but it will not scale gracefully to large DNN weight tensors. A PyTorch model with several million parameters can easily exceed ten megabytes per artifact; at that point the blob column approach introduces non-trivial read amplification for ClickHouse and makes artifact management cumbersome. The appropriate migration path is to a cloud-managed model store once that threshold is crossed.

The Optuna hyperparameter search persists trials across sessions in `algo.hyperparam_trials` and resumes seamlessly on subsequent calls to `forecast-cli tune`. However, only one search space is currently encoded: the DNN architecture space covering six parameters (`n_neurons`, `activation`, `dropout`, `learning_rate`, `batch_norm`, `l1_reg`). The ten binary feature and selector hyperparameters identified by Lago et al. [5] in their §4.3.3, together with the `preprocessing` and `init_method` choices, remain manual decisions set in the config file, because the preprocessor and model constructors do not yet expose the corresponding constructor-level toggles. Until those toggles exist, including them in the Optuna space would create inert search dimensions.

### 1.5.4 Deferred tooling and adoption triggers

Each tool evaluated and not adopted during the MLOps build-out was deferred to a specific, observable trigger rather than to an indefinite future review.

- **MLflow** — deferred until the production experiment-config count exceeds ten. At the current scale, ClickHouse and W&B together cover the audit-log and research-surface roles that MLflow would fill without adding a seventh service to the Compose stack.
- **Kubernetes** — deferred until a second deploy host is justified by availability requirements. The single-host scope means there is exactly one failure domain; the control-plane overhead of Kubernetes is not warranted until workloads must be distributed across multiple hosts.
- **Cloud-managed model store** — deferred until DNN model weights exceed the practical limit for ClickHouse blob storage, taken as approximately ten megabytes per artifact. Below that threshold the `algo.model_artifacts` table keeps artifacts co-located with their experiment rows without requiring a separate storage service.
- **DVC** — deferred for as long as ENTSO-E data is fully replayable from the Transparency Platform API at the required resolution. The combination of `ReplacingMergeTree` deduplication and the `GIT_SHA` column on `algo.experiment_runs` provides sufficient provenance without maintaining a separate binary data store.

## 1.6 Conclusion

Of Zarour's nine MLOps practices, eight are realised end-to-end in algo-repo and one — P9, the feedback loop and drift-triggered retraining — is minimally realised, scoped to two bidding areas behind an opt-in config flag. The quantitative evaluation in §5.2 backs every realised label with measured values: an ML Test Score with a per-section breakdown, provenance-triple coverage validated by a live re-run, drift-detection counts, and DORA delivery throughput (deployment frequency and lead time), all computable on demand from the live system by re-running `scripts/report/extract_metrics.py`. Every realised practice has been mapped in this chapter to a concrete repository artefact: a Prefect deployment, a ClickHouse migration, a GitHub Actions workflow, or a Grafana dashboard. The four tools that remain deferred — MLflow, Kubernetes, a cloud-managed model store, and DVC — are held back not by oversight but by the deliberate single-host MVP scope under which the deferred-cost and current-value trade-off does not yet justify adoption; each deferred tool carries a named, observable adoption trigger so that the decision to adopt it can be revisited against a concrete condition rather than re-litigated from first principles. Together, these nine practices and their supporting artefacts constitute a production ML platform that ingests live market data, refits daily, monitors prediction quality and data drift, and maintains a complete audit trail from every experiment row back to the exact Git commit and data window that produced it.

## .1 Glossary

**LEAR** Lasso Estimated AutoRegressive ensemble for day-ahead electricity price forecasting, the primary baseline model from Lago et al. [5].

**DNN** Deep neural network; in this chapter, a multi-layer perceptron ensemble whose final prediction is the arithmetic mean of `n_dnn_ensemble` replicates trained with distinct random seeds.

**EPF** Electricity price forecasting: the task of predicting day-ahead wholesale electricity prices for a given bidding zone from historical prices and exogenous signals.

**ENTSO-E** European Network of Transmission System Operators for Electricity; the body that operates the Transparency Platform API from which the platform ingests price, load, and generation data.

**Lasso** Least Absolute Shrinkage and Selection Operator, an L1-regularised linear regression that simultaneously shrinks coefficients and performs automatic feature selection.

**TPE** Tree-structured Parzen Estimator, the Bayesian optimisation sampler used by Optuna when running `forecast-cli tune`.

**MLOps** Machine Learning Operations: the set of practices for reliably building, deploying, and operating machine learning systems in production.

**ReplacingMergeTree** A ClickHouse storage engine that deduplicates rows sharing the same sort key during background merge, used throughout the platform to make re-insertion safe under the platform's append-only write semantics.

**Tailscale** A mesh VPN built on WireGuard used here to grant the GitHub Actions runner ephemeral SSH access to the deploy host without exposing any port on that host to the public internet.

**Loki** Grafana Labs' log aggregation system, used here as the structured-log backend for all Prefect flows and infrastructure modules via direct Python push through `python-logging-loki`.

**Evidently** An open-source library for data and model drift detection, used here in in-process mode inside the Prefect drift-detection flow to run a Kolmogorov–Smirnov test comparing a 90-day reference window against a 7-day current window.

Table 1.2: Technology stack and rationale.

| **Tool** | **Role** | **Status** | **Why this choice** |
|---|---|---|---|
| Prefect | Orchestration | Kept | Sufficient for single-host flows; lighter than Airflow or Dagster |
| ClickHouse | OLAP store + model artifact blob | Kept | Columnar performance, ReplacingMergeTree dedup, native Polars / Arrow path |
| Docker Compose | Container runtime + infra-as-code | Kept | Single-host scope; YAML matches the deploy mental model |
| Loki | Log aggregation | Added | Direct `python-logging-loki` push, no Promtail sidecar |
| Prometheus | Metrics time-series store | Added | System metrics, uptime tracking; native ClickHouse and Grafana scrape endpoints, no exporter sidecars |
| Grafana | Dashboards and file-based alerts | Added | File-provisioned configs survive container restarts |
| Evidently AI | In-process drift detection | Added | Mode 1 reports; no separate Evidently service |
| W&B | Research + production tracking | Added | Two projects (`algo-research`, `algo-prod`); `WANDB_MODE=disabled` in CI |
| GitHub Actions | CI/CD | Kept | Native to the GitHub-hosted repo, no self-hosted runner yet |
| Tailscale | Deploy host network access | Added | SSH to deploy host without exposing it to the public internet |
| Optuna | Hyperparameter search (TPE) | Added | Persisted to ClickHouse; revisited per-config rather than per-run |
| MLflow | Experiment registry | Deferred | Until experiment-config count exceeds ten |
| Kubernetes | Container orchestration | Deferred | Until multi-host deployment is justified |
| DVC | Data versioning | Deferred | ENTSO-E data is replayable from source; ClickHouse + `GIT_SHA` suffice |

Table 1.3: Maturity assessment of Zarour's nine practices, with the Google MLOps maturity model level [3] assigned to each.

| **Practice** | **Status** | **Google level** | **One-line evidence** |
|---|---|---|---|
| P1 — Version control | Realised | 2 | Git, GitHub, `configs/`, migrations, dashboards and alert rules all in Git |
| P2 — Continuous integration | Realised | 2 | `pr_test`, `pr_ruff-lint`, `pr_smoke` gates on every PR |
| P3 — Reproducibility | Realised | 2 | `GIT_SHA` plus data-window columns stamped on every experiment run |
| P4 — Containerisation | Realised | 2 | `docker-compose.yml` plus file-provisioned Grafana and alert rules |
| P5 — Collaboration | Realised | 1 | `CLAUDE.md`, `REPO_OVERVIEW.md`, `RUNBOOK.md`, `CONTRIBUTING.md`, plus Slack alerts |
| P6 — Continuous training | Realised | 1 | `daily-forecast` deployments refit at 06:00 UTC |
| P7 — Experiment tracking | Realised | 1 | `algo.experiment_runs` plus W&B research surface |
| P8 — Continuous monitoring | Realised | 2 | Loki plus three Grafana dashboards (Data Quality, Operational, System Health) plus file-based alert rules; model-quality metrics in W&B `algo-prod` |
| P9 — Feedback loop and retraining | Minimal | 1 | Drift-triggered retrain behind a config flag, two areas only |

Table 1.4: ML Test Score self-assessment (min-of-sections rule).

| **Section (max 7)** | **Sum** |
|---|---|
| Data tests | 2.5 |
| Model development tests | 3.0 |
| ML infrastructure tests | 4.0 |
| Monitoring tests | 4.0 |
| **ML Test Score (min)** | **2.5** |

# Bibliography


[1] Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, and D. Sculley. The ML test score: A rubric for ML production readiness and technical debt reduction. In *2017 IEEE International Conference on Big Data (Big Data)*, pages 1123–1132, 2017.

[2] Nicole Forsgren, Jez Humble, and Gene Kim. *Accelerate: The Science of Lean Software and DevOps*. IT Revolution Press, 2018.

[3] Google Cloud. MLOps: Continuous delivery and automation pipelines in machine learning, 2024. Google Cloud Architecture Center whitepaper. Defines a three-level MLOps maturity model (Level 0 manual, Level 1 ML pipeline automation, Level 2 CI/CD pipeline automation).

[4] Odd Erik Gundersen and Sigbjørn Kjensmo. State of the art: Reproducibility in artificial intelligence. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.

[5] Jesus Lago, Grzegorz Marcjasz, Bart De Schutter, and Rafał Weron. Forecasting day-ahead electricity prices: A review of state-of-the-art algorithms, best practices and an open-access benchmark. *Applied Energy*, 293:116983, 2021.

[6] Ananth Mahadevan and Michael Mathioudakis. Cost-aware retraining for machine learning. *Knowledge-Based Systems*, 293:111610, 2024.

[7] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009.

[8] Rakshith Subramanya, Seppo Sierla, and Valeriy Vyatkin. From DevOps to MLOps: Overview and application to electricity market forecasting. *Applied Sciences*, 12(19):9851, 2022.

[9] Mohammad Zarour, Hussein Alzabut, and Khalid T. Al-Sarayreh. MLOps best practices, challenges and maturity models: A systematic literature review. *Information and Software Technology*, 183:107733, 2025.